VIETNAM ACADEMY OF SCIENCE AND TECHNOLOGY
UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

BASIC PROGRAMMING

Lecture 2: Variables, Data Types, Operators

Dr. NGUYEN Hoang Ha
Dr. NGUYEN Minh Huong

Lecture 2: Variables, Data Types, Operators

- Computer Memory
- Variables and Constants
- Basic Data Types
- Operators

Computer Memory

Computer memory

- All programs is loaded into RAM (Random Access Memory) when they are called by CPU
- RAM areas



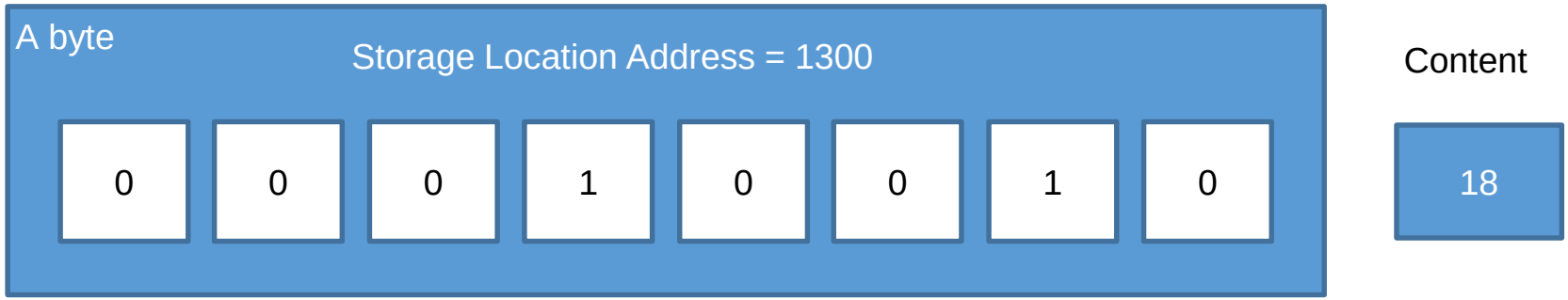
RAM segments

- Code segment: stores compiled code
- Data segment: stores global variables and static variables
- Heap segment: allocates dynamic variables
- Stack segment: stores local variable, function parameters and other function-related information.



How computers store data?

- Bit
- Byte = 8 Bits
- Each byte has been labeled with a number call **address**



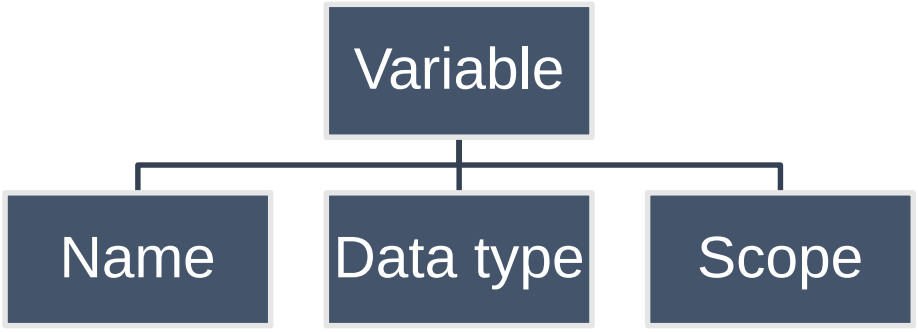
Computer Memory Units

- 1 Byte = 8 bits
- 1 KB = 1024 (= 2^{10}) bytes
- 1 MB = 1024 KB
- 1 GB = 1024 MB
- 1 TB = 1024 GB

Variables and Constants

What is a Variable?

- A named area of computer memory
- Objective: to store values
- Scope of validity: where its content can be accessed and manipulated



Variable Name Rules

- Includes: letters, digits, underscore "_"
  - → excludes: special characters (@, +,-,*,/,#,%,&…)
- First character must be a letter or underscore
- Upper case and lower case are distinct
- Must not be a keywords

Which of following name is valid for C variables

- _student
- 3Students
- B11_student
- Stu-dent
- student@
- Return

Scope of a Variable

```
int z = 38;
int main()
{
    int y = 30;
        while (y < 400)
        {
                int x = 10;
                y += x++;
                y ++;
        }
    z= y *10;
}
```

Scope of z

Scope of y

Scope of x

Variable Declarations

- A variable must be declared before being used
- Declaration statement defines:
  - Name
  - Type
  - Description (if any)

Type name; / comment */*

- Example:

```
float average_gpa_2020_bi11; /* average gpe of Bi11 students*/
```

Variable Initialization

- After declaring a variable, initial value should be set to avoid junk value.
- Syntax: Type name = value;
- Example:

```
int z = 30;
```

Or

```
int z;
z = 30;
```

Variable Assignment

- Change a variable by assignment operation "="
- Syntax: name = value ;
- Example:

```
int z;
z = 30;
z = 90;
z = -19;
```

Basic Data Types

Data Types

Basic types:

- int
- float
- double
- char
- void

Example:

```
int average;
float gpa_2020;
```

Integer variables

- Declaration:

```
int name; /* comments */
```

- An integer is any whole number without a decimal point

Print out integer variables

- Using printf() function (from stdio library)
- %d : integer conversion specification
- Syntax:

printf (format. expression1, expression2,…);

- Example

```
int temp = 10;
printf("twice %d is %d \n", temp, 2*temp);
```

Result:

```
twice 10 is 20
```

Ranges of Integer Variables

int	4bytes	-2147483648 to 2147483647
unsigned int	4bytes	0 to 4294967295
signed int	4bytes	-2147483648 to 2147483647
short int	2bytes	-32768 to 32767
unsigned short int	2bytes	0 to 65,535
signed short int	2bytes	-32768 to 32767
long int	4bytes	-2,147,483,648 to 2,147,483,647
signed long int	8bytes	same as long int
unsigned long int	8bytes	0 to 4,294,967,295
long long int	8bytes	-(2^63) to (2^63)-1
unsigned long long int	8bytes	0 to 18,446,744,073,709,551,615

Integer Constants in Decimal

- int constant

```
const int i = 100 ;
```

- Long const

```
const long i = 100l;
```

- Unsigned int constant

```
const unsigned i = 100u;
```

- Unsigned long constant

```
const unsigned i = 100ul;
```

or

```
const unsigned i = 100lu;
```

Integer Constants in other Numbering System

- Octal int constant:

```
const int temp = 023;
printf("temp = %d \n", temp);
```

Result:

```
temp = 19
```

- Hexa int constant:

```
const int temp = 0x23;
printf("temp = %d \n", temp);
```

Result

```
temp = 35
```

- Binary int constant:

```
const int temp = 0b0101;
```

Result:

```
temp = 5
```

Floating Point Values

- Floating-point numbers:
  - Using decimal point
  - 5.0 is a floating-point number
  - 5 is an integer number

Ranges of Values

Type	Storage size	Value range	Precision
float	4 byte	1.2E-38 to 3.4E+38	6 decimal places
double	8 byte	2.3E-308 to 1.7E+308	15 decimal places
long double	10 byte	3.4E-4932 to 1.1E+4932	19 decimal places

Declaration

- `float temp = 2.5f;`
- `double temp1 = 12e2;`
- `long double temp2 = 1.89L;`

Print out a floating-point number

- Syntax:

%[width].[precision][modifier]f

  - *Width: minimum number of characters to be printed*
  - *Precision: number of digit to be printed after decimal point*
  - *Modifier: add L for printing out long double value*

- Example:

```
float temp = 21.12345;
printf("temp = %3.5f \n", temp);
```

Result:

```
temp = 21.12345
```

Character Variables

- Single byte: one character
- keyword: *char*
- *Example:*

```
char a = 'A';

char b = 65;
```

Representation of a character in C

- By a character in a pair of single quotes:

```
char a = 'A';
```

- By a number in the ASCII table:

```
char a = 65;
```

Print out Characters

- %c
- Example:

```
char temp = 65;
printf("temp = %c \n", temp);
```

Result:

```
temp = A
```

Void Types

- No value
- Use cases:
  - Function return as void
  - Function has void argument
  - Pointer to void

Complex Number Types

- Form: a + bi
- Include header file: <complex.h>

Constants

- A variable with fixed value

```
const float Pi = 3.14159f;//a fixed variable definition so
this occupies memory
```

- Macro is an alternative way to write fixed conventional values using #define:

```
#define PI 3.14159f;//the pre-processor replaces PI with
3.14159f before compilation
```

Notes

- Choose correct data types:
  - Appropriate range of values
  - Correct type
  - Example:

Notes

```c
#include <stdio.h>
int main(void)
{
    const float Revenue_Per_150 = 4.5f;
    short JanSold = 23500; /* Stock sold in January */
    short FebSold = 19300; /* Stock sold in February */
    short MarSold = 21600; /* Stock sold in March */
    float RevQuarter = 0.0f; /* Sales for the quarter */
    short QuarterSold = JanSold+FebSold+MarSold;
    /* Calculate quarterly total */
    /* Output monthly sales and total for the quarter */
    printf("\nStock sold in\n Jan: %d\n Feb: %d\n Mar: %d", JanSold,FebSold,MarSold);
    printf("\nTotal stock sold in first quarter: %d",QuarterSold);
    /* Calculate the total revenue for the quarter and output it */
    RevQuarter = QuarterSold/150*Revenue_Per_150;
    printf("\nSales revenue this quarter is:$%.2f\n",RevQuarter);
    return 0;
}
```

Notes

- Choose correct data types:
  - Appropriate range of values
  - Correct type

Print out Variables of different types

- Print out different types with printf()

Conversion Character	How the corresponding argument is printed
c	as a character
d	as a decimal integer
e	as a floating point number; example 7.123000e+00
f	as a floating point number; example 7.123000
g	in the e-format or f-format, whichever is shorter
s	as a string

Print out Variables of different types

- Special character used in printf()

Special Character	Escape Sequence
alert (beep)	\a
backslash	\\
backspace	\b
carriage return	\r
double quote	\"
formfeed	\f
horizontal tab	\t
newline	\n
null character	\0
single quote	\'
vertical tab	\v
question mark	\?

Define new data type with enumerations

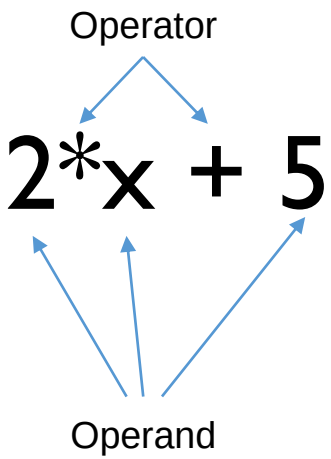
- Enumeration type: list of key words
- Syntax:

  enum Name {value 1, value 2,.., value n};
- Example:

```
enum DayInWeek {Sun = 1, Mon, Tue, Wed, Thu, Fri , Sat};
```

Operators

Operators in C



Operators in C

- Arithmetic operators
- Relational operators
- Logical operators
- Bitwise operators
- Assignment operators
- Miscellaneous operators

Arithmetic Operators

Operator	Meaning of Operator
+	addition or unary plus
-	subtraction or unary minus
*	multiplication
/	division
%	remainder after division (modulo division)

Arithmetic Operators: example

```c
#include <stdio.h>
int main(void)
{
    int a = 21, b = 10;
    c = a + b; printf("Line 1 - Value of c is %d\n", c );
    c = a - b; printf("Line 2 - Value of c is %d\n", c );
    c = a * b; printf("Line 3 - Value of c is %d\n", c );
    c = a / b; printf("Line 4 - Value of c is %d\n", c );
    c = a % b; printf("Line 5 - Value of c is %d\n", c );
    c = a++; printf("Line 6 - Value of c is %d\n", c );
    c = a--; printf("Line 7 - Value of c is %d\n", c );
    return 0;
}
```

Relational Operators

Operator	Meaning of Operator	Example
==	Equal to	5 == 3 is evaluated to 0
>	Greater than	5 > 3 is evaluated to 1
<	Less than	5 < 3 is evaluated to 0
!=	Not equal to	5 != 3 is evaluated to 1
>=	Greater than or equal to	5 >= 3 is evaluated to 1
<=	Less than or equal to	5 <= 3 is evaluated to 0

Relational Operators: example

```c
#include <stdio.h>
int main(void)
{
    int a = 21, b = 10;
    printf("Line 1 - Value of a==b is %d\n", a==b );
    printf("Line 2 - Value of a!=b is %d\n", a!=b );
    printf("Line 3 - Value of a>b is %d\n", a>b );
    printf("Line 4 - Value of a<b is %d\n", a<b );
    return 0;
}
```

Logical Operators

Operator	Description	Example
&&	Called Logical AND operator. If both the operands are non-zero, then the condition becomes true.	(A && B)
||	Called Logical OR Operator. If any of the two operands is non-zero, then the condition becomes true.	(A || B)
!	Called Logical NOT Operator. It is used to reverse the logical state of its operand. If a condition is true, then Logical NOT operator will make it false.	!(A && B)

```c
#include <stdio.h>

int main(void)
{
    int a = 1, b = 0;
    printf("Line 1 - Value of a&&b is %d\n", a&&b );
    printf("Line 2 - Value of a||b is %d\n", a||b );
    printf("Line 3 - Value of !a is %d\n", !a );
    return 0;
}
```

Bitwise Operators

- Syntax & Meaning:

Operator	Meaning
&	Bitwise AND operator
|	Bitwise OR operator
^	Bitwise exclusive OR operator
~	Binary One's Complement Operator is a unary operator
<<	Left shift operator
>>	Right shift operator

- Truth table

x	y	x & y	x | y	x ^ y
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

```c
#include <stdio.h>
int main(void)
{
    unsigned int a = 60; /* 60 = 0011 1100 */
    unsigned int b = 13; /* 13 = 0000 1101 */
    int c = 0; c = a & b; /* 12 = 0000 1100 */
    printf("Line 1 - Value of c is %d\n", c );
    c = a | b; /* 61 = 0011 1101 */
    printf("Line 2 - Value of c is %d\n", c );
    c = a ^ b; /* 49 = 0011 0001 */
    printf("Line 3 - Value of c is %d\n", c );
    c = ~a; /*-61 = 1100 0011 */
    printf("Line 4 - Value of c is %d\n", c );
    c = a << 2; /* 240 = 1111 0000 */
    printf("Line 5 - Value of c is %d\n", c );
    c = a >> 2; /* 15 = 0000 1111 */
    printf("Line 6 - Value of c is %d\n", c );
    return 0;
}
```

Assignment Operators

```c
#include <stdio.h>
int main(void)
{
    int a = 21; int c ; c = a;
    printf("Line 1 - = Operator Example, Value of c = %d\n", c);
    c += a; printf("Line 2 - += Operator Example, Value of c = %d\n", c);
    c -= a; printf("Line 3 - -= Operator Example, Value of c = %d\n", c);
    c *= a; printf("Line 4 - *= Operator Example, Value of c = %d\n", c);
    c /= a; printf("Line 5 - /= Operator Example, Value of c = %d\n", c);
    c = 200;
    c %= a; printf("Line 6 - %= Operator Example, Value of c = %d\n", c);
    c <<= 2; printf("Line 7 - <<= Operator Example, Value of c = %d\n", c);
    c >>= 2; printf("Line 8 - >>= Operator Example, Value of c = %d\n", c);
    c &= 2; printf("Line 9 - &= Operator Example, Value of c = %d\n", c);
    c ^= 2; printf("Line 10 - ^= Operator Example, Value of c = %d\n", c);
    c |= 2; printf("Line 11 - |= Operator Example, Value of c = %d\n", c);
    return 0;
}
```

Misc. Operators

```c
#include <stdio.h>
int main(void)
{
    int a = 4; short b; double c; int* ptr;
    /* example of sizeof operator */
    printf("Line 1 - Size of variable a = %d\n", sizeof(a) );
    printf("Line 2 - Size of variable b = %d\n", sizeof(b) );
    printf("Line 3 - Size of variable c= %d\n", sizeof(c) ); /* example
    of & and * operators */
    ptr = &a; /* 'ptr' now contains the address of 'a'*/
    printf("value of a is %d\n", a);
    printf("*ptr is %d.\n", *ptr);
    /* example of conditional expression */
    a = 10;
    b = (a == 1) ? 20: 30;
    printf( "Value of b is %d\n", b );
    b = (a == 10) ? 20: 30;
    printf( "Value of b is %d\n", b );
    return 0;
}
```

Operator Precedence in C

- Example:

Multiplication operator has a higher precedence than the addition operator

Operator Associativity

- Operators may be associative, meaning the operations can be grouped arbitrarily
- Only needed: when the operators in an expression have the same precedence
- Left-associative: the operations are grouped from the left
- Right-associative: the operations are grouped from the right

Implicit Type Conversion

- When an operator is used with operands of different types, type conversion occurs to make the operands compatible.
- The compiler converts all operands into the data type of the largest operand.

Explicit Type Conversion

- Syntax:

  (data_type)expression;

- Example (from slide 34)

Explicit Type Conversion

```c
#include <stdio.h>
int main(void)
{
    const float Revenue_Per_150 = 4.5f;
    short JanSold = 23500; /* Stock sold in January */
    short FebSold = 19300; /* Stock sold in February */
    short MarSold = 21600; /* Stock sold in March */
    float RevQuarter = 0.0f; /* Sales for the quarter */
    short QuarterSold = JanSold+FebSold+MarSold; /*inappropriated data type, value exceeds ~64000 short datatype range*/
    /* Calculate quarterly total */
    /* Output monthly sales and total for the quarter */
    printf("\nStock sold in\n Jan: %d\n Feb: %d\n Mar: %d", JanSold,FebSold,MarSold);
    printf("\nTotal stock sold in first quarter: %d",QuarterSold);
    /* Calculate the total revenue for the quarter and output it */
    RevQuarter = QuarterSold/150*Revenue_Per_150;
    printf("\nSales revenue this quarter is:$%.2f\n",RevQuarter);
    return 0;
}
```

Explicit Type Conversion

- Explicit type conversion:
  - Convert the value of a variable to another type

```
RevQuarter = (float)QuarterSold/150*Revenue_Per_150;
```

Mathematical Functions

- Include header <math.h>